

Renewable Energy RAG Project

Step-by-Step Code Documentation — Based on the uploaded Jupyter Notebook

Purpose: explain what each code cell in the notebook does, why it is used, what happens to the data, what output is produced, and what the next RAG step is.

1. Complete RAG Pipeline

PDF → PyPDFLoader → Documents → Text Chunks → Ollama Embeddings → Chroma Vector Database → Retriever → Relevant Chunks → Prompt → Ollama LLM → Answer + Source Pages

The uploaded notebook implements this pipeline using a five-page renewable-energy PDF. The notebook output shows 5 loaded pages and 17 chunks after splitting.

2. Step 1 — Import the Required Libraries

Notebook code:

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_ollama import ChatOllama
from langchain_core.output_parsers import StrOutputParser
```

What happens? These imports bring in the components needed for document loading, chunking, embeddings, vector storage, prompt creation, and the Ollama chat model.

Next step: Load the PDF document.

3. Step 2 — Load the PDF

```
pdf_path = r"C:\Users\JayanthJella\Downloads\renewable_energy_guide.pdf"
loader = PyPDFLoader(pdf_path)
documents = loader.load()
print(f"Loaded {len(documents)} pages")
```

What happens? The PDF path is given to PyPDFLoader. The loader reads the PDF and creates LangChain Document objects. In this notebook, the output is **Loaded 5 pages**, meaning five page-level Document objects were created.

Data at this stage: each Document contains page text and metadata such as its page number.

Next step: inspect the loaded documents to confirm that text was extracted correctly.

4. Step 3 — Inspect the Loaded Document

```
print("Number of documents:", len(documents))
print(documents[0].page_content[:1000])
```

What happens? The first line prints the number of loaded documents. The second line displays the first 1,000 characters of the first page. This is a validation step: before creating embeddings, you should make sure the PDF text has actually been extracted.

The notebook output begins with the title **Renewable Energy: A Practical Guide** and content about renewable energy, including major types, benefits, challenges, applications, and questions.

Next step: split the page documents into smaller chunks.

5. Step 4 — Split the Document into Chunks

```
text_splitter=RecursiveCharacterTextSplitter(chunk_size=500,chunk_overlap=50)
chunks=text_splitter.split_documents(documents)
print("Number of chunks:", len(chunks))
```

Why chunking is needed: a complete PDF page or large document may contain more information than is useful for one retrieval operation. Smaller chunks make semantic search more targeted.

chunk_size=500: the splitter aims for chunks of about 500 characters. **chunk_overlap=50:** neighboring chunks share some text so information near a chunk boundary is less likely to be lost.

The notebook output is **Number of chunks: 17**. Therefore, 5 loaded page documents became 17 searchable chunks.

Next step: convert the chunks into numerical embeddings.

6. Step 5 — Create Embeddings

```
embeddings=OllamaEmbeddings(model="nomic-embed-text")
```

What is an embedding? An embedding represents text as a numerical vector. Text with similar meaning should produce vectors that are closer together in vector space.

Here the notebook uses OllamaEmbeddings with the **nomic-embed-text** model. The embedding model will be used when the chunks are inserted into the vector database and later when a user's question is converted into a vector for retrieval.

Next step: store the chunk embeddings and original chunk information in Chroma.

7. Step 6 — Create the Chroma Vector Database

```
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

print("Vector database created.")
```

What happens? Chroma receives the 17 chunks and the embedding model. Each chunk is converted into an embedding vector and stored in the vector database together with the document information/metadata.

persist_directory='./chroma_db' tells Chroma where to persist the database locally.

Output: Vector database created.

Next step: create a retriever that can search this vector database.

8. Step 7 — Create the Retriever

```
retriever=vectorstore.as_retriever(search_kwargs={"k":6})
```

What happens? The vector store is converted into a retriever interface. When a question is supplied, the retriever performs similarity-based search and returns the most relevant chunks.

k=6 means the retrieval configuration asks for up to six relevant chunks for a question. Those retrieved chunks become the context supplied to the LLM.

Next step: load the local Ollama chat model that will generate the answer.

9. Step 8 — Load the Ollama LLM

```
llm=ChatOllama(model="llama3.2", temperature=0)
```

What happens? ChatOllama connects the RAG application to the local **llama3.2** chat model.

temperature=0 requests deterministic/low-randomness generation. This is useful for document question answering because the goal is to stay closely aligned with retrieved context.

Next step: define instructions telling the LLM how to use the retrieved context.

10. Step 9 — Create the RAG Prompt

```
prompt=ChatPromptTemplate.from_template("""
You are a document question-answering assistant.

Answer the question using ONLY the provided context.

If the answer is not available in the context, say:
"I could not find this information in the document."

Do not use outside knowledge.

At the end, mention the page number(s) used as the source.

Context:
{context}

Question:
{question}

Answer:
""")
```

Why this is important: the prompt defines the behavior of the answer-generation stage. The model is explicitly instructed to use only the retrieved context, avoid outside knowledge, and mention source pages.

{context} will later contain the retrieved document chunks. **{question}** will contain the user's question.

Next step: format retrieved documents so their page numbers are visible to the model.

11. Step 10 — Format the Retrieved Documents

```
def format_docs(docs):
    return "\n\n".join(f"[Page{doc.metadata.get('page',0)+1}]\n{doc.page_content}"
                        for doc in docs)
```

What happens? The function receives the retrieved Document objects and joins them into one context string.

The expression **doc.metadata.get('page',0)+1** converts the zero-based page index used by the metadata into a human-readable page number. For example, metadata page 0 is displayed as Page 1.

The result looks conceptually like: **[Page1] relevant text ... [Page2] relevant text ...**. This allows the model to see the source page alongside the retrieved text.

Next step: combine retrieval, context formatting, prompt formatting, and LLM generation in one function.

12. Step 11 — Build the ask_question() Function

```
def ask_question(question):
    retrieved_docs=retriever.invoke(question)
    context= format_docs(retrieved_docs)

    messages=prompt.format_messages(context=context,question=question)
    response=llm.invoke(messages)

    print("\nAnswer:")
    print(response.content)

    print("\nSources:")
    pages=sorted(set(doc.metadata.get("page",0)+1 for doc in retrieved_docs))
    print ("Pages:",pages)
```

Retrieval: `retriever.invoke(question)` searches the vector database for chunks relevant to the question.

Context: `format_docs(retrieved_docs)` converts those chunks into the context string with page labels.

Prompt: `prompt.format_messages(...)` inserts the context and question into the prompt template.

Generation: `llm.invoke(messages)` sends the completed prompt to llama3.2 and receives the response.

Sources: the function extracts page numbers from retrieved documents, removes duplicates with `set()`, sorts them, and prints them.

Note: the commented line `#return response.content` means the function currently prints the answer rather than returning it.

13. Step 12 — First Question Test

```
ask_question("What are the major types of renewable energy")
```

The notebook returns the five types: Solar Energy, Wind Energy, Hydropower, Biomass Energy, and Geothermal Energy. The generated answer says Page 1 and 2, while the separate retrieved-source list prints Pages [1, 3, 4, 5]. This difference is important: the model's written source statement and the programmatically printed retrieved pages are not currently forced to be identical.

14. Step 13 — Second Question Test

```
ask_question("Why is energy storage useful?")
```

The notebook answers that energy storage can hold electricity when supply is high and provide it later when demand is higher or renewable generation is lower. The generated response identifies Page 2, while the program prints the retrieved pages as [1, 2, 4, 5].

15. Step 14 — Question Outside the Document

```
ask_question("What is RAG? And Explain the difference between RAG and LLM?")
```

The notebook returns: "I could not find this information in the document." This behavior follows the prompt instruction to avoid outside knowledge when the answer is not available in the retrieved context.

The output also shows that the retriever still returned pages [1, 2, 4, 5]. Retrieval of chunks does not guarantee that those chunks actually contain an answer. The prompt is therefore important because it tells the LLM not to invent an answer when the retrieved context does not support the question.

16. What Happens When You Ask a Question?

Stage	What happens
1. User question	Example: Why is energy storage useful?
2. Query retrieval	The retriever searches the Chroma vector database for relevant chunks.
3. Context creation	The retrieved chunks are combined and labeled with page numbers.
4. Prompt creation	Context and question are inserted into the ChatPromptTemplate.
5. LLM generation	llama3.2 receives the instructions, context, and question.
6. Answer	The model generates an answer using the supplied context.
7. Sources	Python extracts page numbers from the retrieved documents and prints them.

17. Why Your RAG Can Answer Document Questions

The key idea is that the LLM does not need to search the PDF directly. The PDF has already been transformed into chunks and embeddings and placed in Chroma. At question time, the question is used to retrieve semantically relevant chunks. Those chunks are then supplied to the LLM as context.

In one sentence: Retrieval finds the relevant information; generation turns that information into a natural-language answer.

18. Current Notebook Observations

- The notebook successfully loads 5 PDF pages.
- The 5 page documents are split into 17 chunks.
- The embedding model configured is nomic-embed-text through Ollama.
- Chroma is persisted to ./chroma_db.
- The retriever is configured with k=6.
- The chat model is llama3.2 with temperature=0.
- The prompt instructs the model to use only retrieved context and report source pages.
- ChatOllama is imported twice; one import can be removed.
- StrOutputParser is imported but not used.
- The function currently prints the response rather than returning it because return response.content is commented out.

19. Interview Explanation

"I built a document question-answering system using RAG. First, I load the PDF with PyPDFLoader. Then I split the documents into smaller overlapping chunks using RecursiveCharacterTextSplitter. I convert those chunks into embeddings using Ollama's nomic-embed-text model and store them in Chroma. For a user question, the retriever searches the vector database and returns the most relevant chunks. I format those chunks as context, add them to a prompt, and send the prompt to the llama3.2 model through Ollama. The prompt restricts the model to the provided context and asks it to mention source pages. Finally, the Python function prints the generated answer and the pages associated with the retrieved documents."

20. End-to-End Summary

PDF is the source → PyPDFLoader extracts text → chunking prepares searchable pieces → embeddings convert meaning into vectors → Chroma stores the vectors → retriever finds relevant chunks → format_docs prepares context → ChatPromptTemplate gives instructions → llama3.2 generates the response → Python prints the answer and retrieved source pages.

Documentation basis: the uploaded Renewable_Energy_RAG_Project.ipynb and its recorded cell outputs.